

Laplacian Eigenvector Tokenization

Giving graph nodes a coordinate system the way sines and cosines give one to a line

positional encoding · src.features.laplacian_positional_encoding · 2026-06-23

In one sentence. A transformer needs to tell its tokens apart, but a graph’s node numbers are arbitrary. Laplacian eigenvectors hand each node a short vector of coordinates that depend only on **where it sits in the graph’s shape** — not on how it was numbered — so two relabelled copies of the same graph get the same encoding.

1 Why a graph needs positional encoding at all

A transformer sees a **set of tokens** and lets every token attend to every other. By itself, attention is permutation-invariant: shuffle the tokens and you get the same answer shuffled. For text that is wrong — “dog bites man” $\neq$ “man bites dog” — so transformers add a **positional encoding**: a vector glued to each token that says **where it is**.

For a sentence “where” is easy — position 1, 2, 3, ... along a line, encoded with sines and cosines of different frequencies. For a graph there is no such line. A node has no natural index; the only thing that is real is **the pattern of connections around it**. So the question becomes:

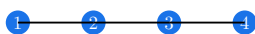
What is the graph-shaped analogue of “sine and cosine of position”?

The answer is the **eigenvectors of the graph Laplacian**. They are, quite literally, the natural vibration modes of the graph — the graph’s own sines and cosines.

2 Building block 1 — the Laplacian matrix

Start from two matrices you already know. For a graph on n nodes:

a path graph $1! - 2! - 3! - 4!$



adjacency A

0	1	0	0
1	0	1	0
0	1	0	1
0	0	1	0

Adjacency A $A_{ij} = 1$ if nodes i and j share an edge, else 0. It records **who connects to whom**.

Degree D a diagonal matrix, $D_{ii} = \sum_j A_{ij}$ = the number of edges at node i . It records **how busy each node is**.

The **combinatorial Laplacian** is simply their difference:

$$L = D - A$$

That tiny formula is doing something specific. Apply L to a vector x that assigns a number x_i to each node, and look at row i :

$$(Lx)_i = D_{ii}x_i - \sum_{j \sim i} x_j = \sum_{j \sim i} (x_i - x_j)$$

So $(Lx)_i$ measures **how different node i ’s value is from its neighbours’** — a discrete second derivative, the graph version of $-\nabla^2$. This is why L is called the Laplacian: on a grid it literally becomes the finite-difference Laplace operator.

2.1 The quadratic form: L measures smoothness

The single most important fact about L is what it computes in the quadratic form $x^\top Lx$:

$$x^\top Lx = \sum_{(i,j) \in E} (x_i - x_j)^2$$

Read this slowly. Hand the graph any assignment of numbers to nodes. Then $x^\top Lx$ is the **total squared disagreement across edges**. It is large when neighbours hold very different values (a jagged signal) and small when neighbours agree (a smooth signal). L is the graph's built-in **roughness meter**.

Because it is a sum of squares, $x^\top Lx \geq 0$ always: L is **positive semi-definite**. Combined with L being real and symmetric, this guarantees everything we need in the next step.

3 Building block 2 — eigenvectors as vibration modes

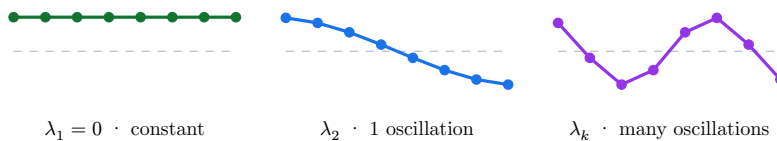
Because L is real, symmetric, and PSD, the spectral theorem gives us a full set of n real eigenvectors $u_1, \dots, u_n$ that are mutually orthogonal, with non-negative eigenvalues

$$0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n.$$

Each eigenvector solves $Lu = \lambda u$. Rearranged through the quadratic form, the eigenvalue **is** the roughness of its eigenvector:

$$\lambda_k = u_k^\top L u_k = \sum_{(i,j) \in E} (u_k(i) - u_k(j))^2 \quad (\|u_k\| = 1).$$

So eigenvectors are ordered **from smoothest to roughest**. This is exactly the physics of a vibrating string or drum: low modes wobble slowly across the whole object, high modes oscillate rapidly. On a path graph the eigenvectors are literally cosine waves of increasing frequency:



The leftmost mode is flat — every node gets the same value. The next splits the graph gently into “one side high, one side low”. Higher modes carve the graph into ever finer alternating regions. **These alternating regions are the coordinates we are after.**

4 Why the first eigenvector is thrown away

The smallest eigenvalue is always $\lambda_1 = 0$, and for a connected graph its eigenvector is the **constant** vector $u_1 = (1, 1, \dots, 1)/\sqrt{n}$. Check it: $L \cdot \mathbf{1} = (D - A)\mathbf{1} = 0$ because each row of A sums to that node's degree.

A constant assigns **the same coordinate to every node**, so it tells you nothing about position. But there is a subtlety the original code missed: a graph with C connected components has eigenvalue 0 with **multiplicity C** , and each of those C modes is a constant-on-one-piece indicator. Skipping only the first column leaves $C - 1$ of them in — and those are literal component-membership flags. The implementation therefore drops **every** eigenvector whose eigenvalue is numerically zero, not just the first:

```
# src/features.py — symmetric normalized Laplacian, drop every zero-eigenvalue mode
eigvals, eigvecs = torch.linalg.eigh(L)                # ascending eigenvalues
non_trivial = (eigvals > 1e-5).nonzero()[0]           # skip ALL ~0 modes (one per component)
pe = eigvecs[:, non_trivial[:k]]                      # keep the next k smallest
```

Aside — counting connected components. The multiplicity of eigenvalue 0 equals the number of connected components. If the graph splits into two pieces there are **two** independent constant-

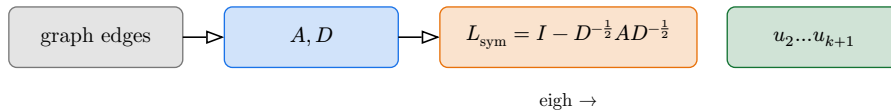
like modes (one per piece). This is the spectral fingerprint of connectivity — the very property the **connectedness** task is about — and a reason the Laplacian spectrum is a natural language for these experiments. The filter above removes this **direct** leak, but as Section 9 shows, a subtler **soft** leak survives it. See [\[\[connectedness-dataset-leak\]\]](#).

5 The normalized Laplacian we actually use

`src/features.py` does not use $L = D - A$ directly. It uses the **symmetric normalized** Laplacian:

$$L_{\text{sym}} = I - D^{-\frac{1}{2}} A D^{-\frac{1}{2}}.$$

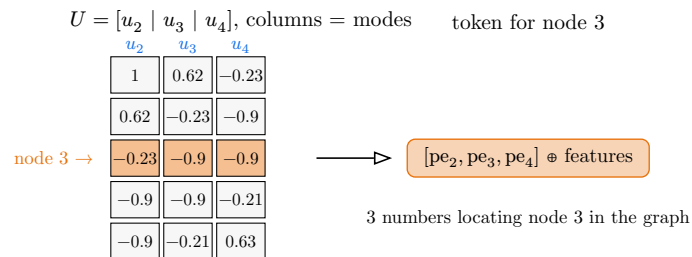
The normalization rescales each node by $1/\sqrt{\text{deg}}$, which keeps high-degree hubs from dominating the spectrum and pins all eigenvalues into the clean range $\lambda \in [0, 2]$. It is the same operator that sits inside a GCN layer, so the positional encoding speaks the same dialect as message passing.



The rest of the story — skip u_1 , keep the next k — is unchanged. The output is an $n \times k$ matrix: **k structural coordinates per node**.

6 From eigenvectors to tokens

Stack the chosen eigenvectors as columns. Reading the matrix **by rows** gives, for each node, its coordinates in the graph’s natural basis:



These k numbers are concatenated onto (or added to) each node token before it enters the transformer. Now attention can distinguish nodes **by their structural position**, and an edge token can tell its two endpoints apart by their coordinates — without ever consulting an arbitrary node index.

6.1 Why this is the encoding the other failures were missing

The companion note on tokenization (`tokenization.typ`) ended with four properties a good node identity must have. Laplacian PE hits all four:

unique within a graph

distinct nodes almost surely get distinct coordinate vectors,

stable across epochs

computed once from the graph, never trained — no gradient fight, coordinates come from this graph’s own spectrum, not a global table,

not shared across graphs

depends only on edges, so a relabelled copy gets the identical encoding.

structure-only

That last point is the crucial one and deserves a proof sketch.

6.2 Permutation equivariance — the property we actually wanted

Relabel the nodes with a permutation P . Then $A \mapsto PAP^\top$ and $L \mapsto PLP^\top$. If $Lu = \lambda u$, then

$$(PLP^\top)(Pu) = PLu = \lambda(Pu),$$

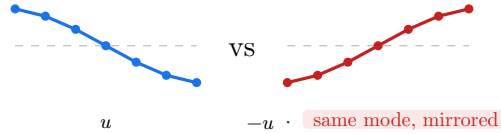
so the eigenvectors simply get **permuted the same way the nodes did**. Node i 's coordinates travel with node i no matter what number you paint on it. This is exactly the **structure-only** property that adjacency-row tokens lacked — there, relabelling produced completely different tokens for the same shape.

7 The one real catch: sign and basis ambiguity

Eigenvectors are not perfectly unique, and this is the thing that trips people up.

7.1 Sign flips

If u is a unit eigenvector, so is $-u$ — they solve $Lu = \lambda u$ equally well. The solver picks a sign **arbitrarily**, and it may pick differently for the same graph on another run or another machine.



Both encode the identical structure, but a model that memorized “ $pe_2 > 0$ means left side” breaks when the sign flips. Standard fixes:

Random sign flipping during training, multiply each eigenvector by a random ± 1 . The model is forced to learn features invariant to sign — the cheap, common default.

Sign-invariant networks feed both u and $-u$ through a shared net and combine symmetrically (SignNet), so the output cannot depend on the choice.

7.2 Repeated eigenvalues

When $\lambda_k = \lambda_{k+1}$ the eigenvectors are only defined **up to rotation within that eigenspace** — there is no canonical choice at all, not even up to sign. Highly symmetric graphs (cycles, grids) have many such repeats. This is a deeper ambiguity than sign flips and is the main theoretical wrinkle of Laplacian PE; sign-flip augmentation handles the common case, and basis-invariant architectures handle the rest.

Practical defaults baked into `src/features.py`. (1) Use L_{sym} so eigenvalues live in $[0, 2]$. (2) Always skip u_1 . (3) Keep a small k (the lowest, smoothest modes carry the most global structure); pad with zeros when the graph has fewer than $k + 1$ nodes. (4) Apply random sign flips at train time. With these four habits, Laplacian PE is a robust, permutation-aware coordinate system for graph tokens.

8 A worked micro-example

Take the path $1! - !2! - !3$. Its Laplacian and the two non-trivial modes:

$$L = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix}, \quad u_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix}, \quad u_3 = \frac{1}{\sqrt{6}} \begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix}.$$

Read the rows to get each node’s 2-D coordinate (we keep $k = 2$):

node	$pe_1 = u_2$	$pe_2 = u_3$	reading
1	+0.71	+0.41	an end node — extreme on the slow mode
2	0.00	−0.82	the centre — zero on u_2 , the dip of u_3
3	−0.71	+0.41	the other end — mirror of node 1

Node 2 sits at the **centre of mass** of the slow mode ($pe_1 = 0$) while the two ends are pushed to opposite extremes — the encoding has discovered the geometry of the path with no coordinates ever supplied.

Notice also that nodes 1 and 3 share pe_2 but differ in pe_1 : it takes **both** coordinates together to separate the symmetric endpoints, which is exactly why we keep several eigenvectors rather than one.

8.1 A gallery of spectra

A few canonical graphs make the “smooth-to-rough” ordering concrete and expose the degeneracy that causes the basis ambiguity of §7. Eigenvalues below are for the **combinatorial** $L = D - A$; the normalized L_{sym} rescales them into $[0, 2]$ but keeps the same qualitative shape.

graph on n nodes	Laplacian eigenvalues	what it teaches
path P_n	$2 - 2 \cos(k\pi/n)$, distinct	cosine modes, no degeneracy — the clean case of §6
cycle C_n	$2 - 2 \cos(2\pi k/n)$, in equal pairs	sine/cosine modes come in degenerate pairs $\rightarrow$ basis rotation, no canonical choice
complete K_n	0, then n with multiplicity $n - 1$	maximal symmetry $\rightarrow$ maximal degeneracy; every non-trivial mode equivalent
star $K_{1,n-1}$	0, $1(\times (n - 2))$, n	one hub mode at the top, a flat band of leaf modes in the middle
two blobs, no bridge	0 with multiplicity 2	disconnected — the multiplicity is the component count
two blobs, one bridge	0, then a tiny $\lambda_2 > 0$	connected but barely — the soft leak of Section 9

The last two rows are the whole subject of the next section: the only difference between them is whether λ_2 is **exactly** zero or merely **near** zero — and that single bit is what the connectedness task asks for.

9 Why Laplacian encoding carries a connectivity signal

Empirically, a one-layer, width-4 transformer reaches **91.5%** on `connectedness_hard` using **only** Laplacian features — even with all zero-eigenvalue modes filtered out. That is far too easy for a model that size if the task truly required tracing reachability. The reason is that the Laplacian spectrum is, by construction, a **soft min-cut detector**: the encoding hands the model the answer geometrically. There are two distinct leaks.

9.1 Leak 1 — the hard leak (multiplicity of zero)

As the aside in §4 noted, the multiplicity of eigenvalue 0 equals the number of connected components. A disconnected graph has ≥ 2 zero modes, each a constant-on-one-component indicator. Reading any one of them off tells you the graph is disconnected immediately. The eigenvalue filter (`eigvals > 1e-5`) exists precisely to delete these indicator columns. This leak is **closed**.

9.2 Leak 2 — the soft leak (algebraic connectivity)

What survives the filter is more interesting. The smallest **non-zero** eigenvalue λ_2 is the graph’s *algebraic connectivity*, or **Fiedler value**. Two facts make it a connectivity oracle:

$\lambda_2 > 0 \Leftrightarrow$ **connected**

the graph is connected **iff** its second eigenvalue is strictly positive,

$\lambda_2 \approx 0 \Leftrightarrow$ **a sparse cut exists**

a graph that is **almost** split into two pieces has an **almost-zero** λ_2 .

The second fact is the *Cheeger inequality*. Writing h_G for the conductance — the cost of the cheapest cut, relative to the smaller side’s volume —

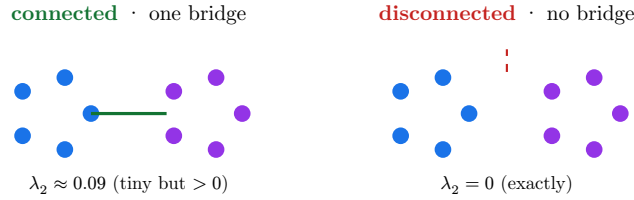
$$\frac{\lambda_2}{2} \leq h_G \leq \sqrt{2\lambda_2}.$$

So connectivity is **not** a knife-edge property in the spectrum; it is a smooth one. “Two dense blobs joined by a single edge” has a cut of cost 1 over a large volume, hence a tiny h_G , hence a tiny λ_2 — and its Fiedler vector u_2 is an **approximate** component indicator: nearly constant on each blob, flipping sign across the bridge.

This is exactly how spectral clustering finds communities: threshold the Fiedler vector and you get the min cut. The Laplacian encoding is spectral clustering’s input — so it carries the cut for free.

9.3 The `connectedness_hard` worst case

The dataset is engineered to sit right on this fault line. Every graph is two dense blobs; the **only** difference between the classes is one edge:



Real spectra from a 5 + 5 instance (normalized L_{sym} , the operator in code), showing the smallest five eigenvalues and the Fiedler vector u_2 :

class	smallest eigenvalues	Fiedler vector u_2 (blob A blob B)
connected	0, 0.086 , 0.85, 0.95, 1.0	+.30 + .25 + .37 + .32 + .32 - .23 - .31 - .37 - .35 - .31
disconnected	0, 0 , 1.0, 1.0, 1.0	0 0 0 0 0 - .43 - .43 - .50 - .43 - .43

Read the table and the whole effect is visible:

- **Connected.** One zero mode, then $\lambda_2 = 0.086$ — a near-zero Fiedler value. Its eigenvector is a clean **step function**: every blob-A node positive, every blob-B node negative. A soft component indicator, handed to the model as the first kept feature column.
- **Disconnected.** **Two** zero modes. After the filter deletes both, the first surviving eigenvector sits at $\lambda = 1$ — a genuine within-blob oscillation, supported on one blob and exactly zero on the other.

9.4 Why the filter does not save us

Deleting the exact-zero modes was supposed to remove the connectivity signal. It does not, for two compounding reasons:

1. **The leading kept column changes character.** Because the disconnected graph loses **two** columns and the connected graph loses **one**, the first surviving eigenvector is a within-blob mode ($\lambda \approx 1$) for the disconnected class but the bridge-spanning Fiedler step ($\lambda \approx 0.09$) for the connected class. The model reads two visibly different leading features.
2. **The Fiedler vector is a soft indicator.** Even ignoring the ordering shift, the connected class’s first kept column is a $\pm$ step across the two blobs — almost as informative as the hard indicator the filter removed. Mean-pooling the sign pattern already separates the classes.

The takeaway for the thesis. The model’s 91.5% is **not** evidence of reasoning through reachability. The Laplacian encoding is a soft spectral cut detector, and `connectedness_hard` differs between classes by exactly the sparsest possible cut — one edge. The embedding does the work; the transformer only reads a coordinate. This is the same “good embedding, not reasoning” pattern as the degree shortcut in `[[connectedness-dataset-leak]]`, one level deeper. To test genuine reasoning the structural answer must be kept **out** of the input features — e.g. honest edge-token tokenization with no positional encoding, or CoT scratchpad tokens where computation must unroll over depth rather than be read off a precomputed spectrum.

10 Summary

Concept	What to remember
Laplacian L	$D - A$; its quadratic form $x^\top Lx = \sum_{ij \in E} (x_i - x_j)^2$ measures roughness
Eigenvectors	the graph's natural vibration modes, ordered smooth $\rightarrow$ rough by eigenvalue
Drop u_1	the constant mode (eigenvalue 0) carries no position; multiplicity = # components
L_{sym}	normalized variant used in code; eigenvalues in $[0, 2]$, matches GCN
Token	each node's row across the kept eigenvectors = its structural coordinates
Equivariance	relabel the graph and the encoding permutes with it — structure-only
Ambiguity	sign flips and degenerate eigenspaces; fix with sign-flip augmentation / SignNet
Connectivity leak	λ_2 (Fiedler) detects sparse cuts via Cheeger; the encoding is a soft min-cut detector, so high connectedness accuracy $\neq$ reasoning

Laplacian eigenvector tokenization is, at heart, one idea: **let the graph tell you where its nodes are, in its own basis of vibration modes**, instead of imposing arbitrary numbers from outside. That is precisely the permutation-aware, structure-only identity the earlier tokenizations were reaching for and missing.